

[Whitehat Disclosure] **Critical**

Vulnerability —

— Responsible Disclosure (Non-Destructive Proof)

Hello [REDACTED] Security Team,

We're reporting an **ultra-critical** vulnerability in the [REDACTED] smart contract that presents a **real and immediate risk of COMPLETELY fund extraction currently valued at 400.000\$**.

This report contains on-chain evidence. We did not execute any transactions to cause damage to the project or to profit as blackhats. We're a professional group of whitehat hackers with a strong background in cybersecurity and extensive web3 experience in bug bounty programs.

TL;DR

A missing validation in [REDACTED] allows users to claim **unlimited** rewards from the pool due to no validation on the smart contract. **Immediate mitigation:** pause `withdraw / claim / [REDACTED]` until a patch is deployed.

1) Short summary

While performing read-only analysis we found repeated withdrawal attempts to the same wallet with no blocking behavior. After examining [REDACTED] Apps (starting with [REDACTED]), we identified a server-side replay vulnerability that permits repeated withdrawal of rewards — effectively allowing unlimited drains. No destructive tests were performed.

2) Why this is urgent

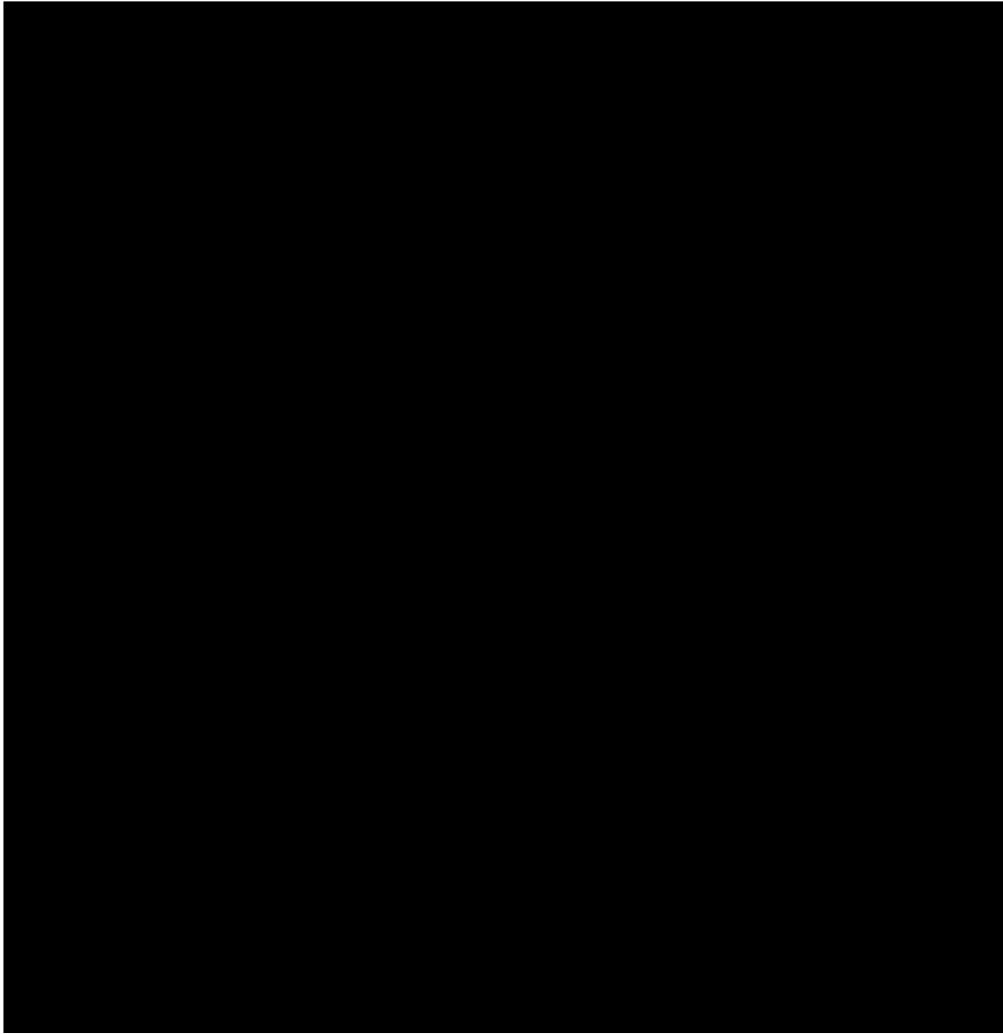
- **Impact:** ~400.000\$ on [REDACTED] ([REDACTED] of [REDACTED])
 - **Exploitability:** Extremely High — Everyone can do it with low technical skills.
 - **Likelihood:** Extremely High — Profit extremely high and no investment needed.
 - **Required skill:** Low — Exploitable with standard hardware; a full drain is achievable by our team in <1 hour (no destructive testing performed).
 - **Conclusion:** High likelihood of rapid exploitation if unpatched and disclosed publicly.
-

3) Reproduction steps

1. Create a [REDACTED] account or in any other App/Dapp that has the same verification process.
2. Watch a [REDACTED] that rewards with [REDACTED] Wait until it's completed and prepare a tool like BurpSuite to capture HTTPs connections.
3. When you complete the [REDACTED] an automatically [REDACTED] will be send to the server and you will get an unsigned tx.
4. Your device will sign it and will send it to the server with the header [REDACTED] and with the signature, this will be processed onchain. This is the normal tx:



5. We execute a replay attack by resending the identical HTTPS request used in the previous transaction via an **automated script**, producing multiple valid on-chain transactions included in the same block or different.



The following attack can be executed unlimited times sending HTTPs requests [redacted] every tx).

4) Estimated impact

- Affected parties: [redacted]
- Funds at risk: $\approx 400.000\$$.
- [redacted]



5) Emergency mitigations (recommended, in order)

1. **Pause** critical state-changing functions (`withdraw`, `claim`, [REDACTED]) immediately.
 2. If pause is unavailable, **revoke external privileged callers**.
 3. Add strict `require` checks for reward claims.
-

6) Suggested on-chain solution

- When [REDACTED] we suggest to add a parameter on the [REDACTED] a boolean `reward_claimed = False` and when users complete the [REDACTED] and claim the first reward `reward_claimed = True` and later if they try to reply attack the [REDACTED] function the pay wouldn't be possible.
-

7) Offer to collaborate

We can help with:

- Full audit of the contract logic and servers from all the [REDACTED] looking for more complex vulnerabilities.
- Running additional **non-destructive** proofs to help the team to fix the problem.
- Technical support from our team with on-chain and off-chain infrastructure.